



On the Ability and Limitations of Transformers to Recognize Formal Languages

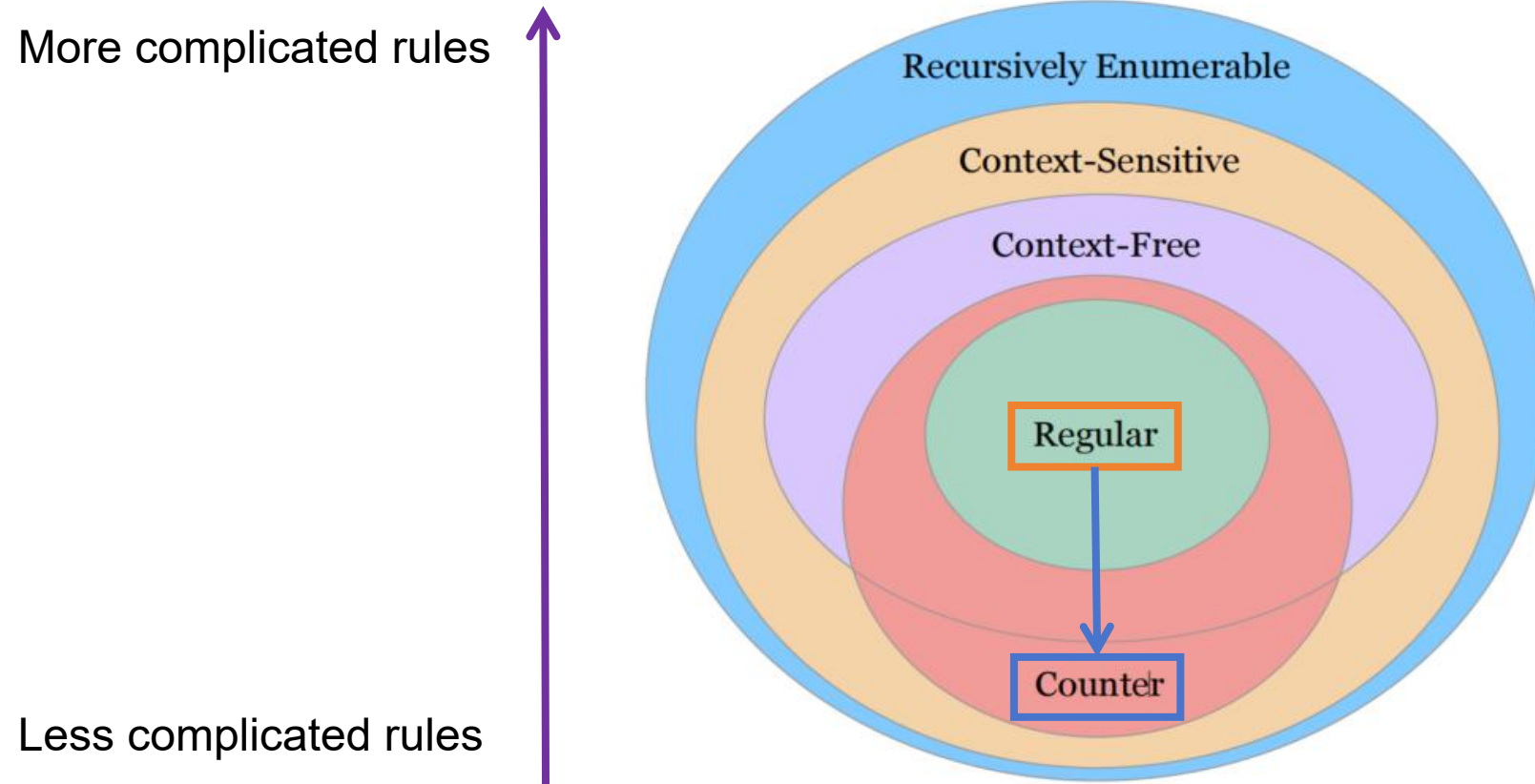
Satwik Bhattamishra[♣] Kabir Ahuja^{◇*} Navin Goyal[♣]

[♣] Microsoft Research India

[◇] Udaan.com

Background

1-Formal Languages



A variant of Chomsky Hierarchy

Background

1-1 Counter Languages

- **Dyck-k**

- **Dyck-1**

- e.g. $((()())())$, $\times (())$

- **Dyck-2**

- e.g. $([()])$, $(([[]]))$, $\times ([()])$

- **Shuffle-Dyck**

Independent counting tasks
without nested structures

- **Shuffle-2**

- e.g. $([])$, $[(([]))$, $\times])[($

- **Shuffle-4**

- **Shuffle-6**

- **n-ary Boolean Expressions**

- **BoolExp-3**

- e.g. $(a \vee b) \wedge c$

- **BoolExp-5**

- e.g. $(\neg a \wedge b) \vee (c \wedge d) \vee e$

- **Other Examples**

- $a^n b^n$

- $a^n b^n c^n$

- $a^n b^n c^n d^n$

Background

1-2 Regular Languages

- **Dot-depth Hierarchy**

Use Boolean closure and nested concatenations to express the language in a star-free regular expression.

- **Dot-depth 0**

e.g. $\{a\}$, $\{\emptyset\}$

- **Dot-depth 1**

e.g. $\{a,b\}^*d(b,c)^*$

- **Dot-depth 2**

e.g. $\{0,1,2\}^*02^*$

- **Dn**: A self-defined language to better research the dot-depth

$$(ab)^* = (b\emptyset^c + \emptyset^c a + \emptyset^c a \emptyset^c b \emptyset^c + \emptyset^c b b \emptyset^c)^c$$

- **Star-free**

- $aa^*bb^*cc^*dd^*ee^*$
- $\{a,b\}^*d(b,c)^*$
- $\{0,1,2\}^*02^*$
- D2/D3/D4/D12

- **Non-star-free**

- Parity

Determine the parity of the number of elements in a sequence

e.g. 11,111

- $(aa)^*$
- $(aaaa)^*$
- $(abab)^*$

Background

1-2 Regular Languages

- **Tomia Grammars** $\rightarrow \Sigma = \{0,1\}$

- **Tomia-1**

- $\rightarrow 1^*$

- **Tomia-2**

- $\rightarrow (10)^*$

- **Tomia-3**

- $\rightarrow$ Odd number of consecutive 1s followed by an even number of 0s

- e.g. 11100, $\times 110$

- **Tomia-4**

- $\rightarrow$ Not contain 000

- e.g. 1001, $\times 10001$

- **Tomia-5**

- $\rightarrow$ An even number of 0s and 1s

- e.g. 1100, $\times 10$

- **Tomia-6**

- $\rightarrow (N(1) - N(0)) \% 3 == 0$

- e.g. 111, 110, $\times 10$, $\times 1110$

- **Tomia-7**

- $\rightarrow 0^*1^*0^*1^*$

Related Work

2-LSTM

LSTM Networks Can Perform Dynamic Counting

Mirac Suzgun¹ Sebastian Gehrmann¹ Yonatan Belinkov^{1,2} Stuart M. Shieber¹

¹ Harvard John A. Paulson School of Engineering and Applied Sciences

² MIT Computer Science and Artificial Intelligence Laboratory
Cambridge, MA, USA

{msuzgun@college, {gehrmann, belinkov, shieber}@seas}.harvard.edu

Abstract

In this paper, we systematically assess the ability of standard recurrent networks to perform dynamic counting and to encode hierarchical representations. All the neural models in our experiments are designed to be small-sized networks both to prevent them from memorizing the training sets and to visualize and interpret their behaviour at test time. Our results demonstrate that the Long Short-Term Memory (LSTM) networks can learn to recognize the well-balanced parenthesis language (Dyck-1) and the shuffles of multiple

(Hochreiter and Schmidhuber, 1997), a popular variant of RNNs, can, theoretically, emulate a simple real-time k -counter machine, which can be described as a finite state controller with k separate counters, each containing integer values and capable of manipulating their content by adding ± 1 or 0 at each time step (Fischer et al., 1968). The authors further tested their theoretical result by training the LSTM networks to learn $a^n b^n$ and $a^n b^n c^n$. Their examination of the cell state dynamics of the models exhibited the existence of simple counting mechanisms in the cell states. Nonetheless,

On the Practical Computational Power of Finite Precision RNNs for Language Recognition

Gail Weiss
Technion, Israel

Yoav Goldberg
Bar-Ilan University, Israel

Eran Yahav
Technion, Israel

{sgailw, yahave}@cs.technion.ac.il
yogo@cs.biu.ac.il

Abstract

While Recurrent Neural Networks (RNNs) are famously known to be Turing complete, this relies on infinite precision in the states and unbounded computation time. We consider the case of RNNs with finite precision whose computation time is linear in the input length. Under these limitations, we show that different RNN variants have different computational

vector that corresponds to a prefix of the sequence and which can be passed as input for a subsequent feed-forward prediction network operating in constant time. The amount of tape used by a Turing machine under this restriction is linear in the input length, reducing its power to recognition of context-sensitive language. More importantly, computation is often performed on GPUs with 32bit floating point computation, and there is increasing evidence that competitive performance can be achieved also for quantized networks with

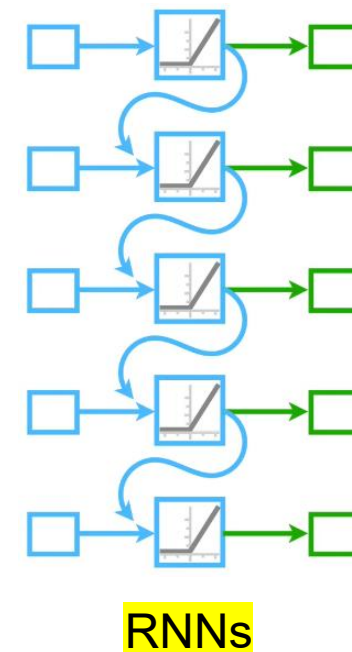
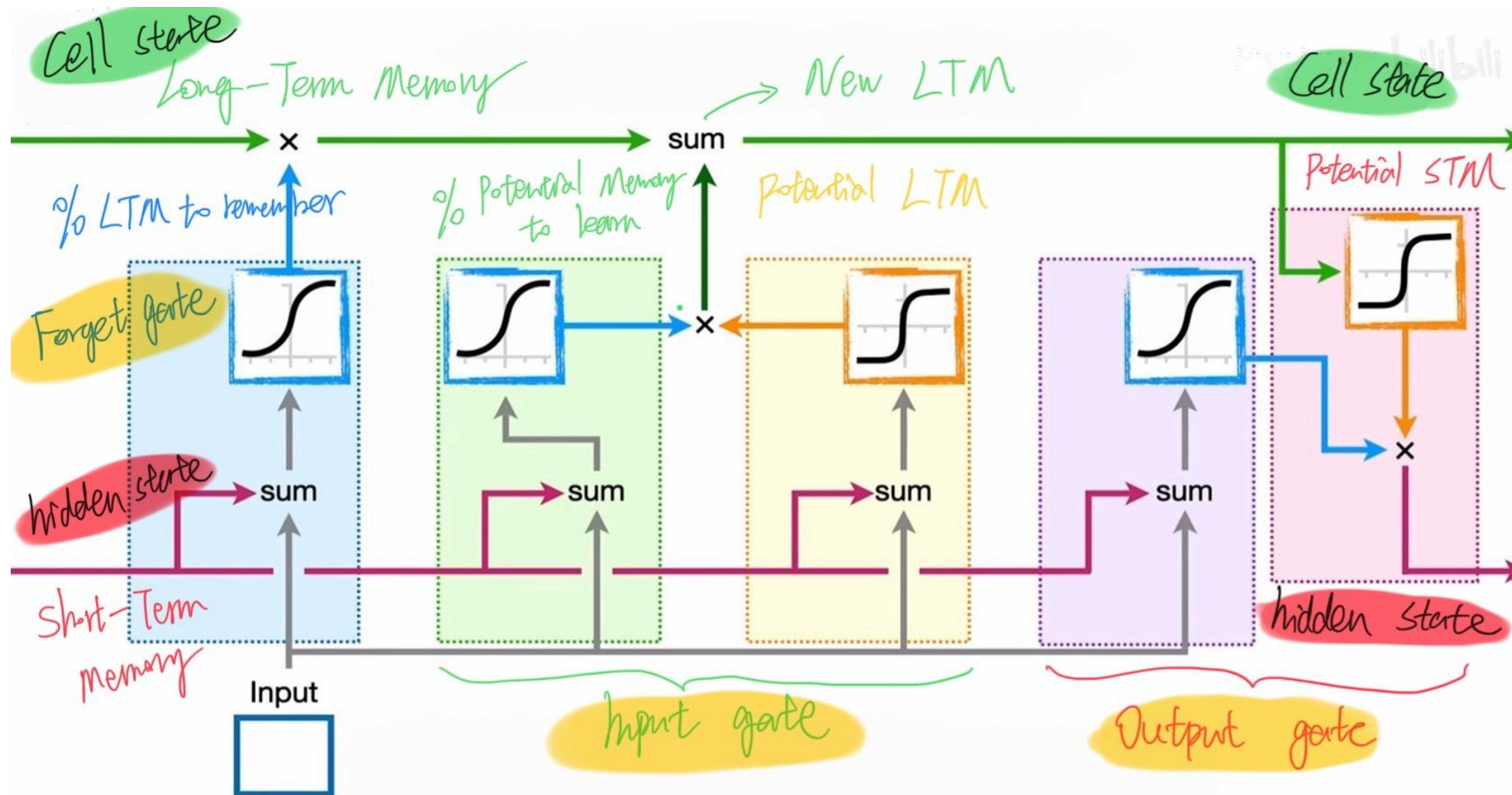
L] 9 Jun 2019

13 May 2018

[1906.03648] LSTM Networks Can Perform Dynamic Counting

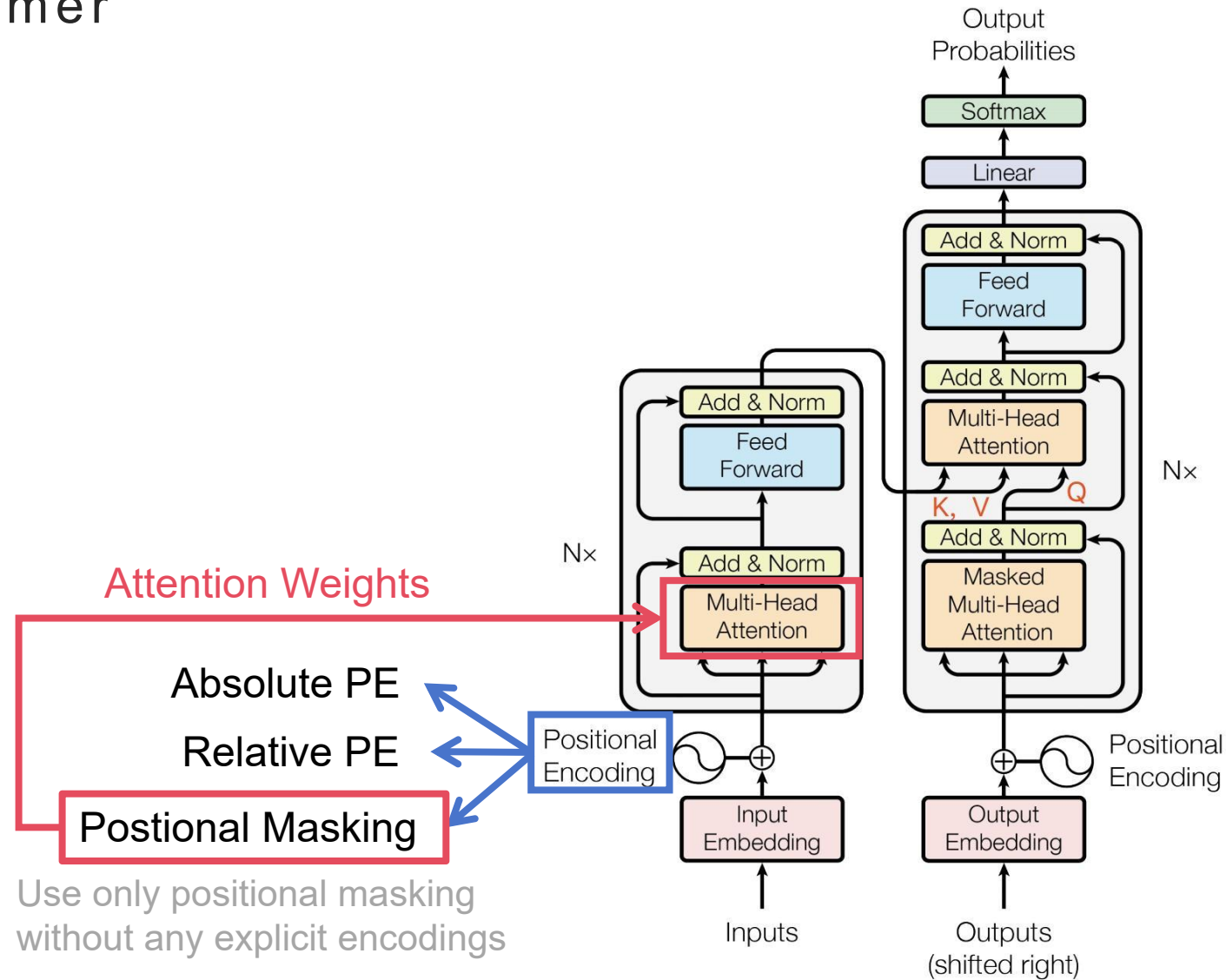
[1805.04908] On the Practical Computational Power of Finite Precision RNNs for Language Recognition

Background 2-LSTM



Background

3-Transformer



Construction of Self-Attention

- **Shuffle-Dyck**

Section 4.1 & Appendix (Lemma C.1)

- $N("[") \geq N("]")$
- **Self-Attention block**
 - $\begin{matrix} [{}_j \rightarrow +1 \text{ at } 2j \\ \rightarrow -1 \text{ at } 2j+1 \end{matrix} \left. \vphantom{\begin{matrix} [{}_j \rightarrow +1 \text{ at } 2j \\ \rightarrow -1 \text{ at } 2j+1 \end{matrix}} \right\} \text{rest indices} = 0$
 - $\begin{matrix}]_j \rightarrow -1 \text{ at } 2j \\ \rightarrow +1 \text{ at } 2j+1 \end{matrix} \left. \vphantom{\begin{matrix}]_j \rightarrow -1 \text{ at } 2j \\ \rightarrow +1 \text{ at } 2j+1 \end{matrix}} \right\} \text{rest indices} = 0$

e.g. Dyck-1

$$[\rightarrow [+1, -1]^T$$

$$] \rightarrow [-1, +1]^T$$

- **Uniform Attention Weights and aggregation**
- **Relu**

- **Depth-Length ratio**

$$ai = \left[\frac{\sigma([{}_j) - \sigma(]_j)}{i}, \frac{\sigma(]_j) - \sigma([{}_j)}{i} \right]^T$$

$\xrightarrow{\text{nesting depth}} = \sigma([{}_j) + \sigma(]_j)$

- **n-ary Boolean Expressions**

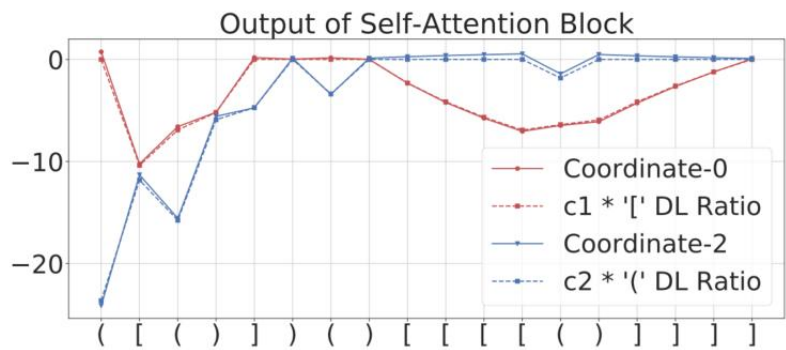
Appendix (Lemma C.2)

- $[+(r-1), -(r-1)] \quad r \rightarrow \text{arity}$

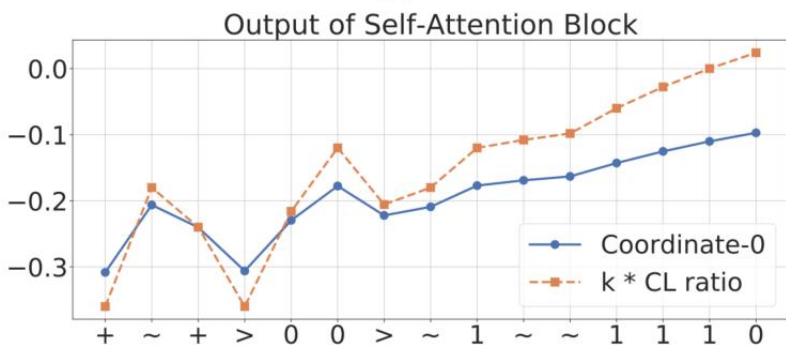
- **Counter-Length ratio:**

$$ai = \left[\frac{c_j}{i}, -\frac{c_j}{i} \right]$$

Results on Counter Languages

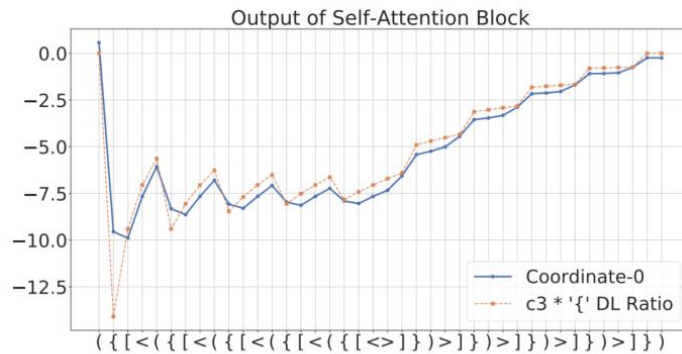


(a)

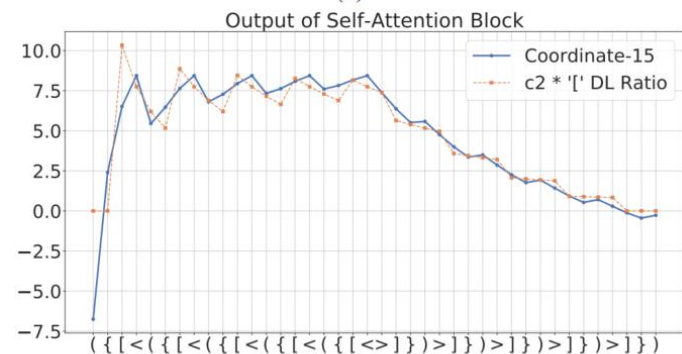


(b)

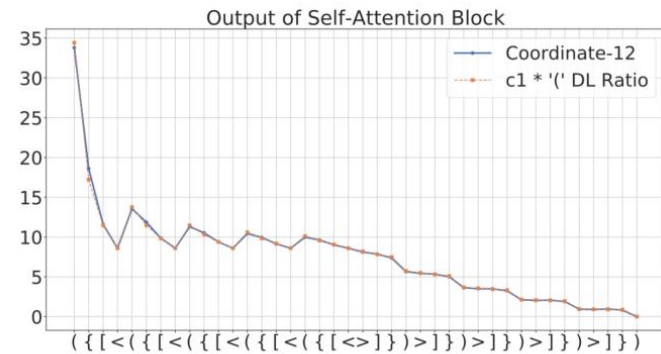
Figure 2: Values of different coordinates of the output of self-attention block of the models trained on Shuffle-2 and BoolExp-3. The dotted lines are the scaled depth to length ratios for Shuffle-2 and scaled counter value to length ratios for BoolExp-3. We observe a near perfect Pearson correlation coefficient of 0.99 between outputs of self attention block and the DL and CL ratios.



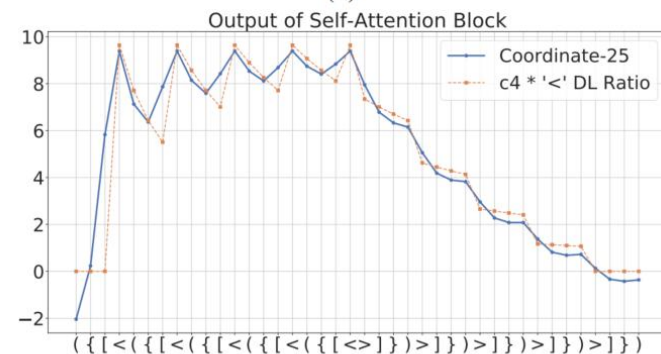
(a)



(c)



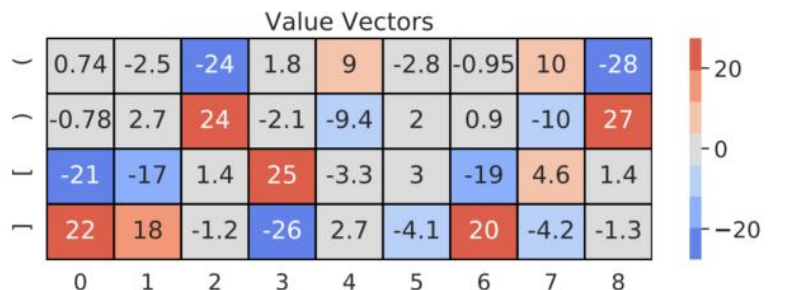
(b)



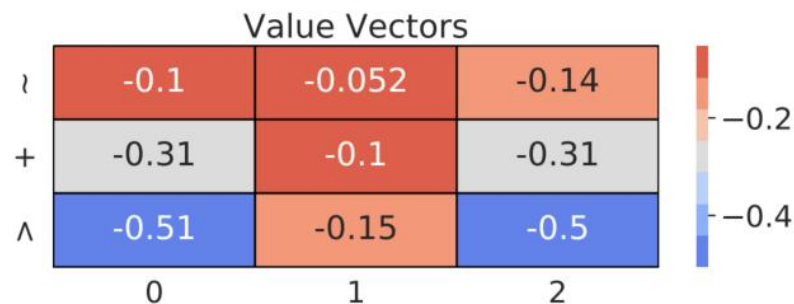
(d)

Figure 5: Values of four different coordinates of the output of self-attention block. The model is trained to recognize Shuffle-4. The dotted lines are the scaled depth to length ratio for the four types of bracket provided for reference.

Results on Counter Languages

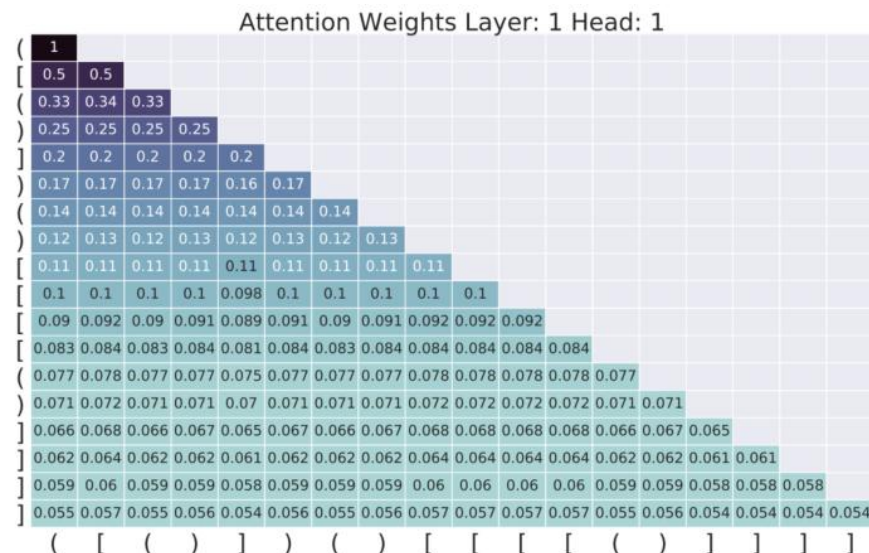


(a)

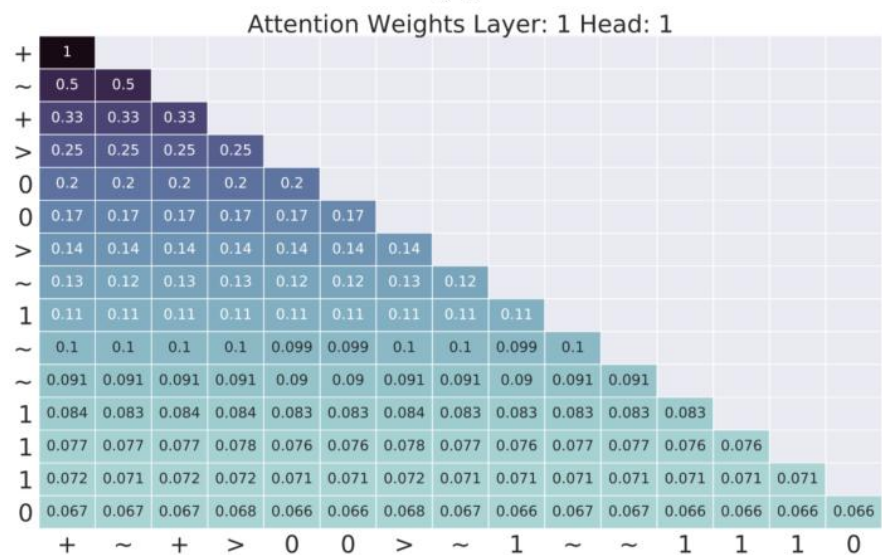


(b)

Figure 3: Plot of value vectors of transformer based models trained on Shuffle-2 3a and Boolean-3 language 3b. The Shuffle-2 model had a hidden size of 8 and boolean-3 model had a hidden size of 3. The x-axis corresponds to different components of the value vectors for both models. Shuffle-2 language consisted of square and round brackets, while for Boolean-3 we considered 3 operators namely: $\sim$ a unary operator, $+$ a binary operator and finally, $>$ which is a ternary operator..



(a)



(b)

Figure 4: Attention maps for models trained on Shuffle-2 and Boolean-3 languages. Similar to our constructions for recognizing these languages, we observe nearly uniform attention weights in both cases

Transformer vs. LSTM in Counter Languages

Language	Model	Bin-1 Accuracy [1, 50]↑	Bin-2 Accuracy [51, 100]↑	Bin-3 Accuracy [101, 150]↑
Shuffle-2	LSTM (Baseline)	100.0	100.0	100.0
	Transformer (Absolute Positional Encodings)	100.0	85.2	63.3
	Transformer (Relative Positional Encodings)	100.0	51.6	3.8
	Transformer (Only Positional Masking)	100.0	100.0	93.0
BoolExp-3	LSTM (Baseline)	100.0	100.0	99.7
	Transformer (Absolute Positional Encodings)	100.0	90.6	51.3
	Transformer (Relative Positional Encodings)	100.0	96.0	68.4
	Transformer (Only Positional Masking)	100.0	100.0	99.8
$a^n b^n c^n$	LSTM (Baseline)	100.0	100.0	97.8
	Transformer (Absolute Positional Encodings)	100.0	62.1	5.3
	Transformer (Relative Positional Encodings)	100.0	31.3	22.0
	Transformer (Only Positional Masking)	100.0	100.0	100.0

The number of layers

- **Reset-Dyck-1 ((#))**
 - **Single-layer :(**
 - no ability to multiple iterations or layered process information before
 - no special mechanism to erase memories
 - **Multi-layers :)**
 - change weight layer upon layer
 - extract information from local features to global ones:
 - the lower level -> implement the counter-like operation
 - the higher level -> indirectly empty the states to finish the correction/reset operation

Model Type	1-Layer		2-Layer	
	Bin 0	Bin 1	Bin 0	Bin 1
Positional Masking	45.1	38.9	100.0	99.2
Positional Encoding	55.8	37.9	100.0	99.6
LSTM	100.0	100.0	100.0	100.0

Table 2: Results on language Reset-Dyck-1 with different number of layers.

Transformer vs. LSTM in Regular Languages

Language	Star-Free	Transformer		LSTM	
		Bin 0	Bin 1	Bin 0	Bin 1
Tomita 1	✓	100.0	100.0	100.0	100.0
Tomita 2	✓	100.0	100.0	100.0	100.0
Tomita 3	✗	75.4	10.8	100.0	100.0
Tomita 4	✓	100.0	92.4	100.0	100.0
Tomita 5	✗	29.3	0.0	100.0	100.0
Tomita 6	✗	88.8	0.0	100.0	100.0
Tomita 7	✓	100.0	100.0	100.0	100.0

Table 3: Results on Tomita grammar

Language	Property	Transformer		LSTM	
		Bin 0	Bin 1	Bin 0	Bin 1
Parity	non-SF	68.7 (23.0)	0.0 (0.0)	100.0	100.0
$(aa)^*$	non-SF	100 (1.3)	0.0 (0.0)	100.0	100.0
$(abab)^*$	non-SF	100.0 (9.9)	5.4 (0.0)	100.0	100.0
$\mathcal{D}_1$	depth-1	100.0	100.0	100.0	100.0
$\mathcal{D}_2$	depth-2	74.6	3.1	100.0	100.0
$\mathcal{D}_4$	depth -4	90.2	3.3	100.0	100.0

Table 4: Results on non-star-free languages (non-SF) and the language $\mathcal{D}_n$. The values in parenthesis correspond to the scores obtained for a model without residual connections. This is to prevent the model from solving the task by memorizing the positional encodings and study the ability of self-attention mechanism to solve the task.

Postional Encodings

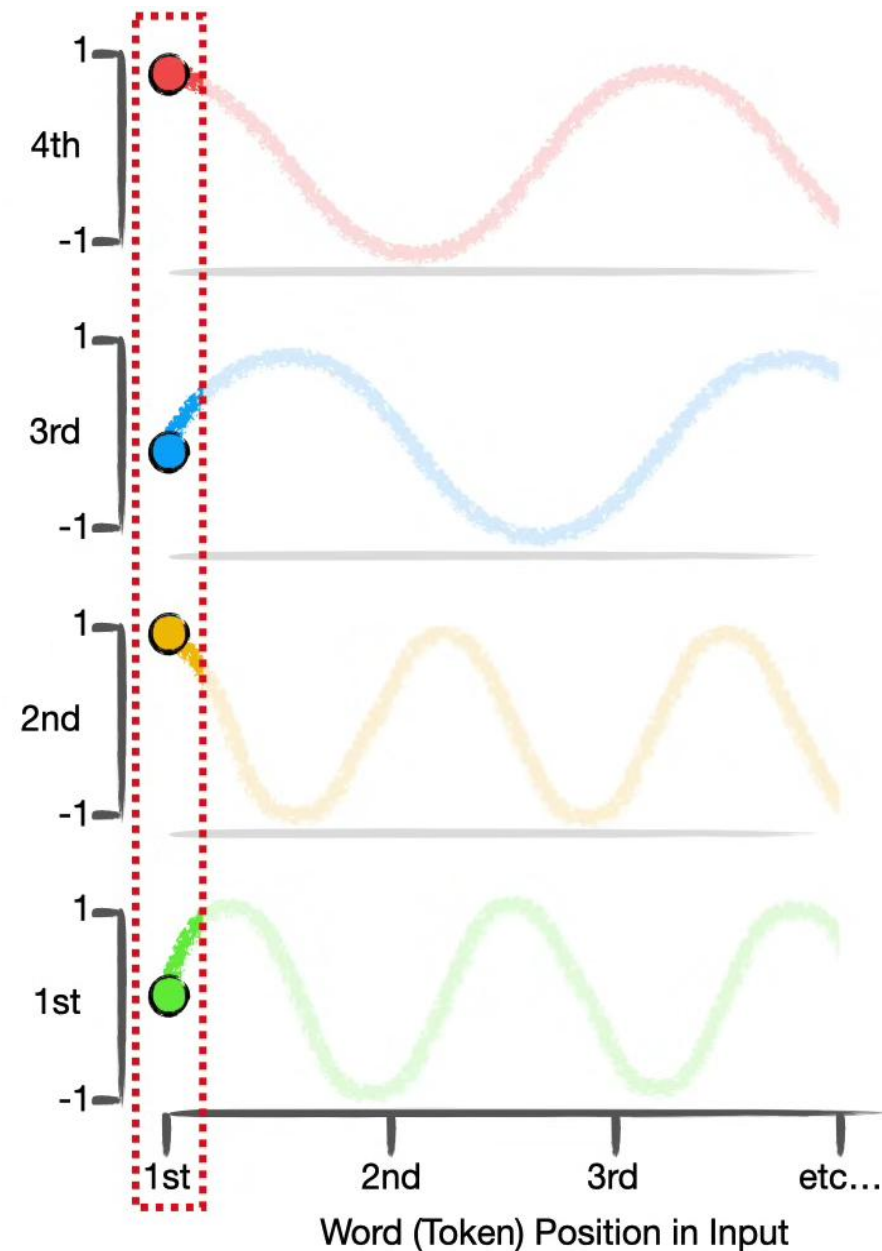
- Absolute PE

- Strictly distinguish sequence positions ✓
- Generalization of higher lengths ↓
 - Not train on some of PE at test time
 - Overfitting

- Relative PE

- Postional Masking

- Generalization ↑
 - Self-attention: More flexibility to adjust attention weights
learn how to process PE by the model itself
- Distinguish sequence positions ×
 - > Appendix Lemma C.4
 - Have Limit to distinguish between odd and even steps
 - Require knowing the relative positions of symbols
e.g. (aa)*, parity



Customed Positional Encoding

- Improve Transformer's **periodicity** capability

- Custom PE-> **$(\cos(n\pi))$**

- **Location1: $\cos(\pi \cdot 1) = \cos(\pi) = -1$**

- **Location2: $\cos(\pi \cdot 2) = \cos(2\pi) = 1$**

- $(aa)^*$ -> period=2

- $(aaaa)^*$ -> **period=4** -> **$\cos((\pi/2)n)$**

- Nothing improves even with

- The increased number of layers
- Different embeddings
- Hyperparameter tuning

Encoding Scheme	$(aa)^*$		$(aaaa)^*$	
	Bin 0	Bin 1	Bin 0	Bin 1
Positional Masking	0.0	0.0	0.0	0.0
Absolute Encoding	1.3	0.0	6.7	0.0
Relative Encoding	0.6	0.0	1.7	0.0
$\cos(n\pi)$	100.0	100.0	0.0	0.0
Trainable Embedding	100.0	0.0	100.0	0.0

Table 5: Performance of transformer based models on $(aa)^*$ and $(aaaa)^*$, for different types of position encoding schemes. To separately study the effect of different position encodings on the self attention mechanism, we do not include residual connections in the models studied here.

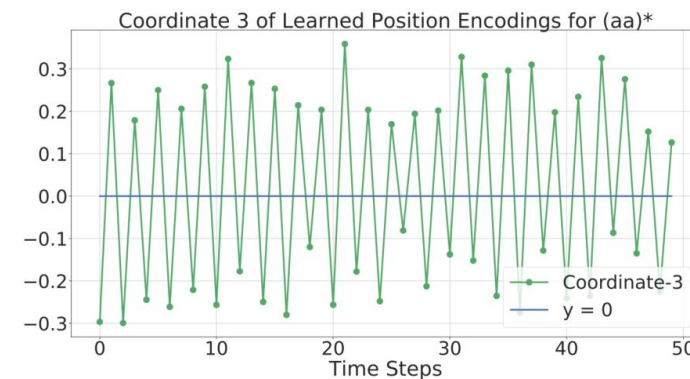
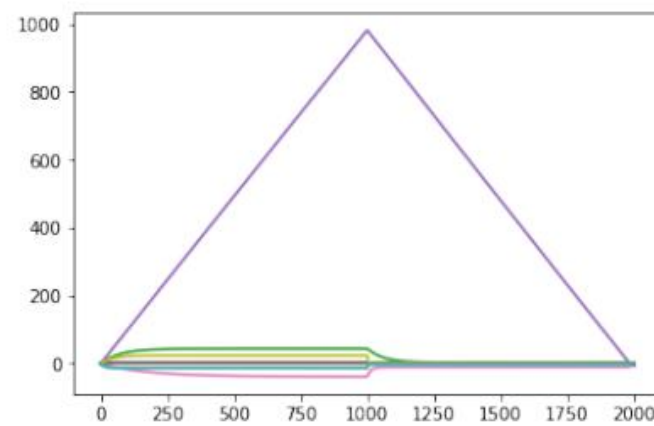


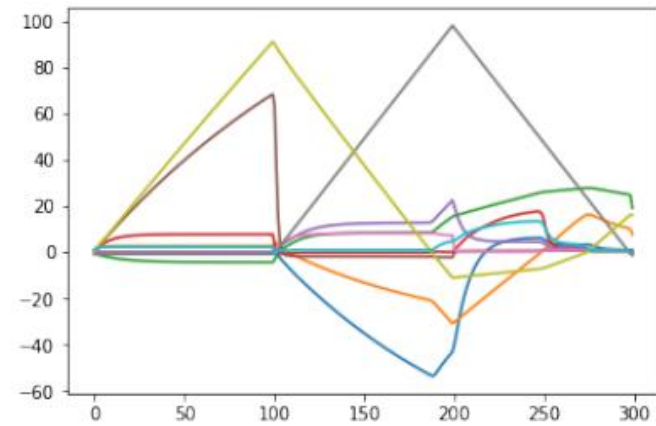
Figure 6: The values of coordiante 3 of the learned position encodings on the language $(aa)^*$. The variation in the encodings resemble a periodic behaviour similar to $\cos(n\pi)$



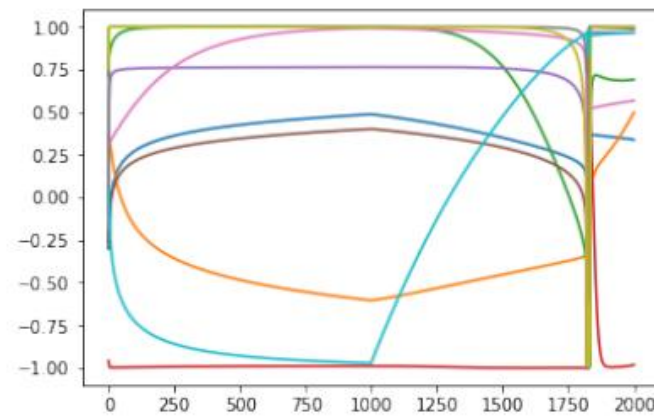
The Ability and Limitations of LSTM to Recognize Formal Languages



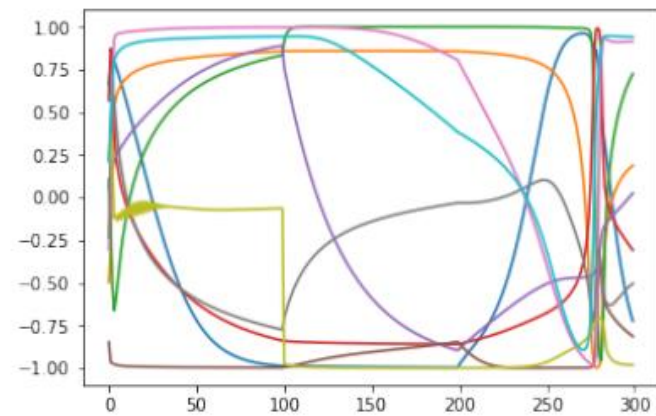
(a) $a^n b^n$ -LSTM on $a^{1000} b^{1000}$



(b) $a^n b^n c^n$ -LSTM on $a^{100} b^{100} c^{100}$



(c) $a^n b^n$ -GRU on $a^{1000} b^{1000}$



(d) $a^n b^n c^n$ -GRU on $a^{100} b^{100} c^{100}$

Figure 1: Activations — c for LSTM and h for GRU — for networks trained on $a^n b^n$ and $a^n b^n c^n$. The LSTM has clearly learned to use an explicit counting mechanism, in contrast with the GRU.

The ability of LSTM in shuffle-Dyck

- 1. Dynamic Counting

- Configuration:
 - 1-layer, <10 dimensions
 - Dyck-1: 3
 - Dyck-2 : 4
 - Shuffle-2 : 4
 - Shuffle-6 : 8
 - Min/Max/Med accuracy

- 2. Baseline model compared to Transformer

- Try for 40 configurations
- the mean of the top 5 accuracies

Task	Model	Training Set			Short Test Set			Long Test Set		
		Min	Max	Med	Min	Max	Med	Min	Max	Med
Dyck-1	RNN	0.45	76.17	46.96	0.28	73.62	41.89	0.06	24.44	7.19
Dyck-1	GRU	99.37	100	100	99.34	100	100	67.68	95.58	84.38
Dyck-1	LSTM	100	100	100	100	100	100	99.98	100	100
Shuffle-2	RNN	33.68	87.77	58.16	29.42	86.48	55.37	0.54	25.58	2.75
Shuffle-2	GRU	99.73	100	99.97	99.62	99.98	99.93	83.70	95.18	93.12
Shuffle-2	LSTM	100	100	100	100	100	100	96.84	99.98	99.35
Shuffle-6	RNN	0.26	57.39	44.54	0.16	54.80	41.38	0	0.64	0.15
Shuffle-6	GRU	96.32	99.98	99.78	96.08	99.98	99.81	51.96	97.30	85.14
Shuffle-6	LSTM	99.68	100	100	99.74	100	100	82.92	99.72	98.14
Dyck-2	RNN	4.37	31.67	14.74	2.96	27.46	12.21	0	0.46	0.01
Dyck-2	GRU	7.78	53.34	28.71	5.38	49.06	25.08	0	1.56	0.05
Dyck-2	LSTM	19.76	52.13	35.82	16.58	48.24	31.29	0	1.46	0.20

Hyperparameter	Bounds
Hidden Size	[3, 32]
Heads	[1, 4]
Number of Layers	[1, 2] — [1, 4]
Learning Rate	[1e-2, 1e-3]
Position Encoding Scheme	[Absolute, Relative, Positional Masking]

Table 7: Different hyperparameters and the values considered for each of them. Note that certain parameters like Heads and Position Encoding Scheme are only relevant for Transformer based models and not for LSTMs. We considered upto 4 layers transformers in the cases where the training accuracies obtained were low for single and two layered networks and reported the results accordingly.

The ability of LSTM in shuffle-Dyck

• 1. Dynamic Counting

- Provide short sequence training data
- Shuffle-2: [2,50]*10,000 -> short: [2,50]*5000; long: [52,100]*5000
- Shuffle-6: [2,50]*30,000 -> short: [2,50]*5000; long: [52,100]*5000

Sample	(()) ()							([()]) [([])]														
Input	(	(	)	)	(	)	(	)	(	[	(	)	]	)	[	(	[	]	)	]	[	]
Output	1	1	1	0	1	0	1	0	1	2	1	2	1	0	2	1	2	1	2	0	2	0

Table 1: Example input-output pairs for the Dyck-1 (left) and Dyck-2 (right) languages.

Sample	([()] [([])])											[{ () }] < [] >										
Input	(	[	(	)	)	]	[	(	[	]	]	)	[	{	(	)	}	]	<	[	]	>
Output	1	3	3	3	2	0	2	3	3	3	1	0	2	6	7	6	2	0	8	24	8	0

Table 2: Example input-output pairs for the Shuffle-2 (left) and Shuffle-6 (right) languages.

• 2. Baseline model compared to Transformer

- Provide longer sequence training data
- Shuffle-2: [2,50]*10,000 -> [2,150]([2,50][51,100][102,150])*2000
- Shuffle-6: [2,1000]*10,000 -> [2,200]*2000

Language	Training Data		Test Data			
	Size	Length Range	Size per Bin	Length Range	Number of Bins	Bin Width
Counter Languages						
Shuffle-2	10000	[2, 50]	2000	[2, 150]	3	50
Shuffle-4	10000	[2, 100]	2000	[2, 200]	3	50
Shuffle-6	10000	[2, 1000]	2000	[2, 200]	3	50
Boolean-3	10000	[2, 50]	2000	[2, 150]	3	50
Boolean-5	10000	[2, 50]	2000	[2, 150]	3	50
$a^n b^n$	50	[2, 100]	50	[2, 300]	3	100
$a^n b^n c^n$	50	[3, 150]	50	[3, 450]	3	150
$a^n b^n c^n d^n$	50	[4, 200]	50	[4, 600]	3	200
Dyck-1	10000	[2, 50]	2000	[2, 150]	3	50

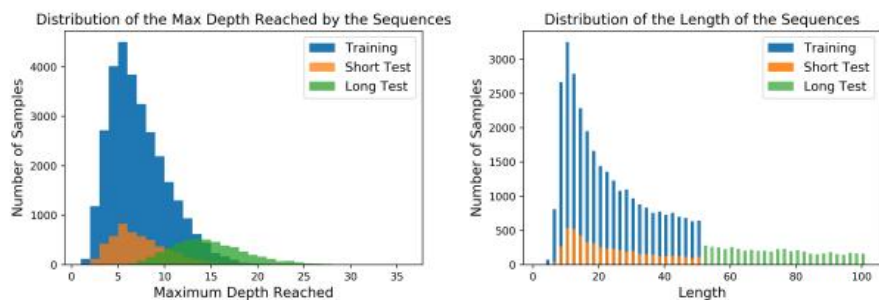


Figure 1: Length and maximum depth distributions of training/test sets for an example Shuffle-6 experiment.

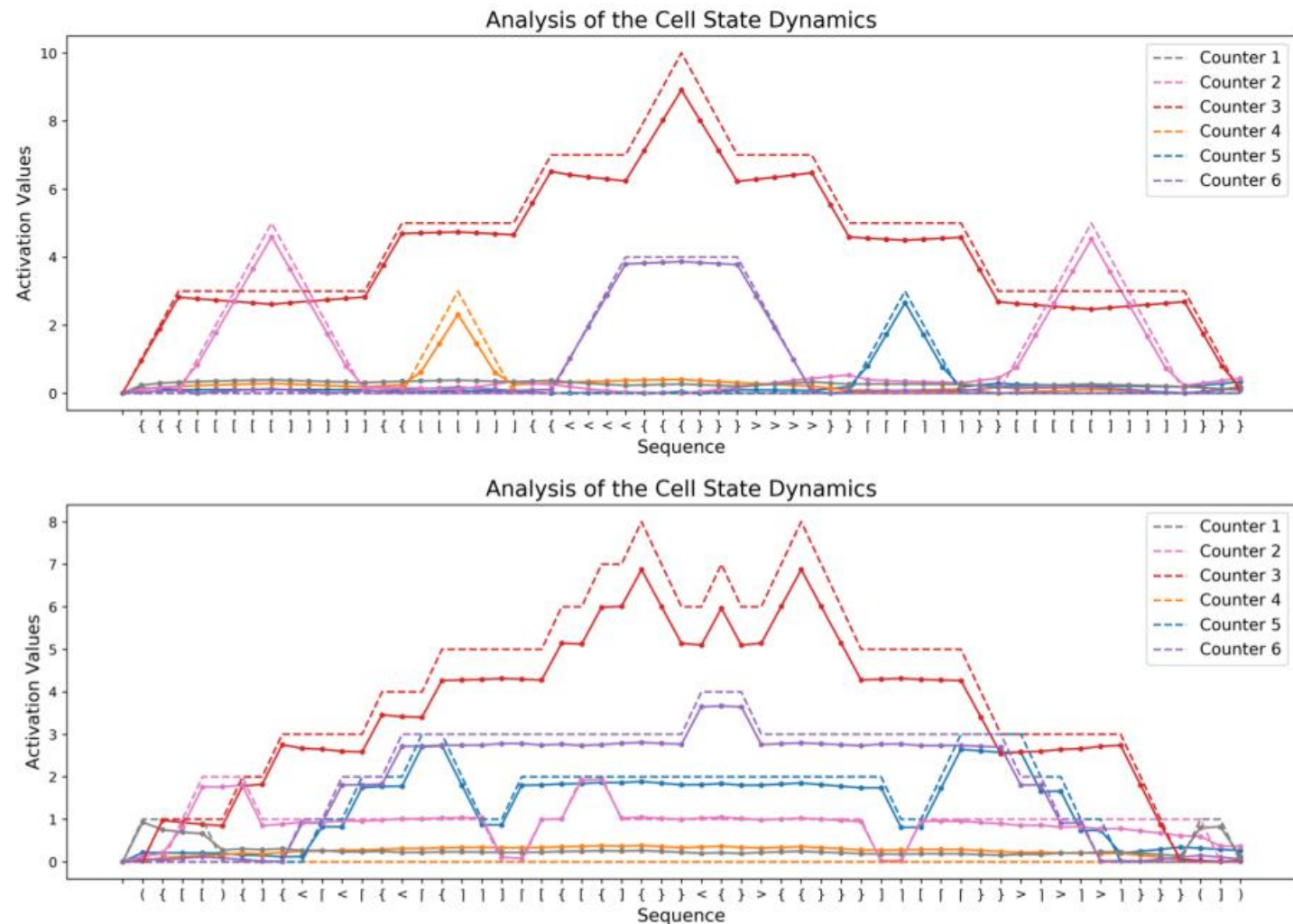
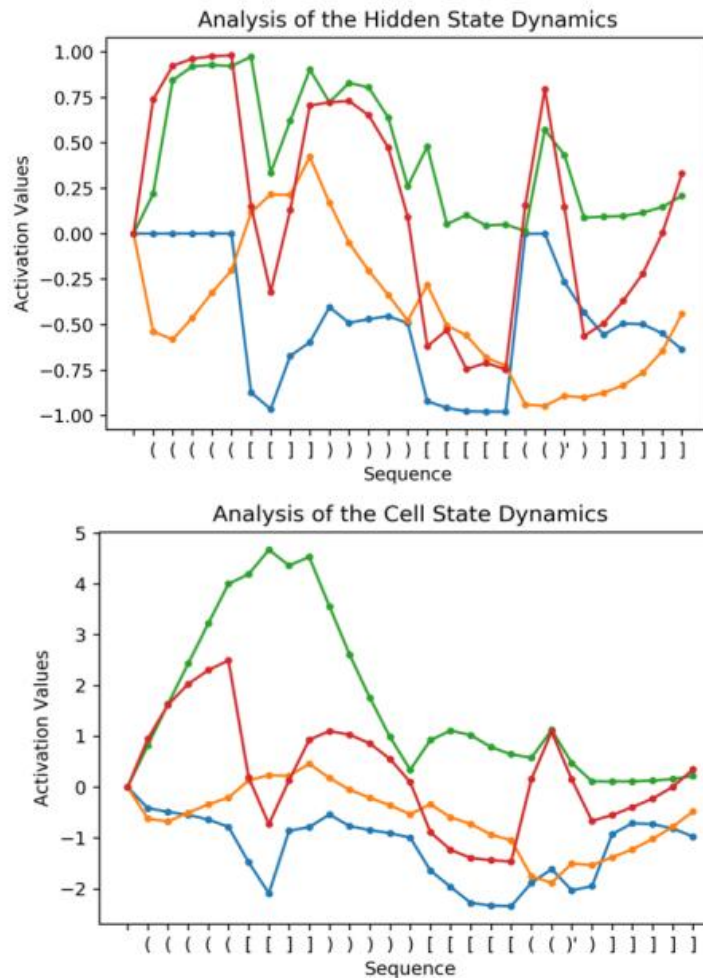
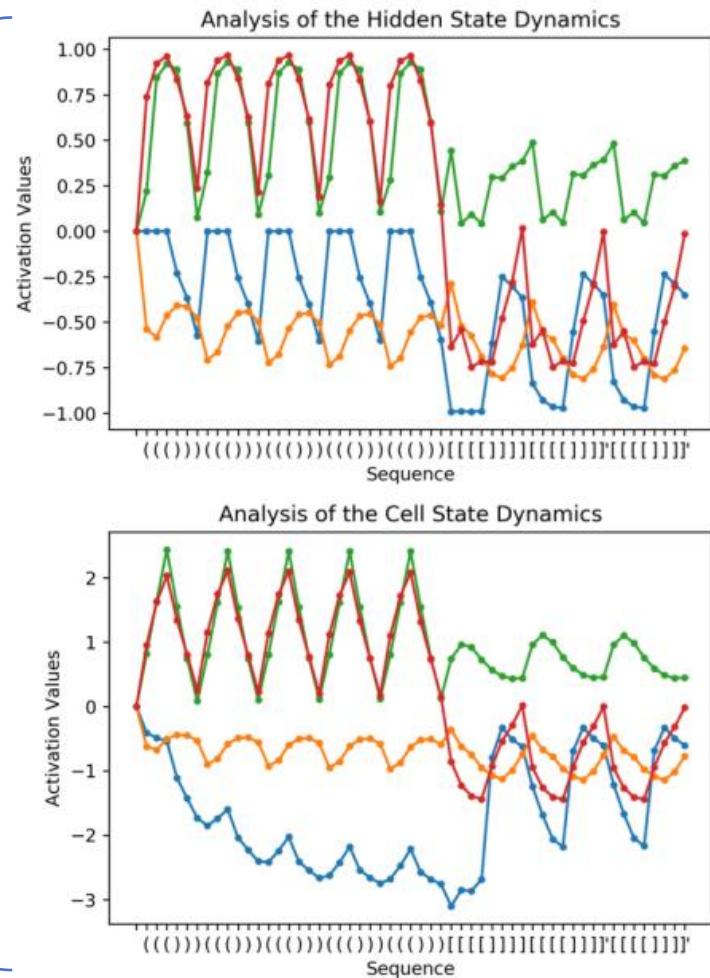


Figure 2: Visualization of the cell state dynamics of one of the LSTM models trained to learn $D_1||^6$, the Shuffle-6 language. The solid lines show the values of the cell states of the six out of eight units in the model, whereas the dashed lines depict the current depth of each distinct parenthesis-pair in $D_1||^6$. We highlight the striking parallelism between the solid lines and the dashed-lines. Our visualizations confirm that the LSTM models employ a simple counting mechanism to recognize the Shuffle languages.

LSTM Networks Can Perform Dynamic Counting

Long seq
Simple counting



Single layer,
4 hidden units

Nested
structure

Complex
counting

Figure 3: Visualization of the hidden and cell state dynamics of one of the LSTMs trained to learn the Dyck-2 language. The time steps at which the model made incorrect predictions are marked with an apostrophe in the horizontal axis. The plots on the left provide a demonstration of the periodic behaviour of the hidden and cell states of the model for a long sequence. Similarly, the plots on the right provide the complex counting behaviour of the model as it observes a nested sequence. We witnessed similar behaviours in our other models as well.

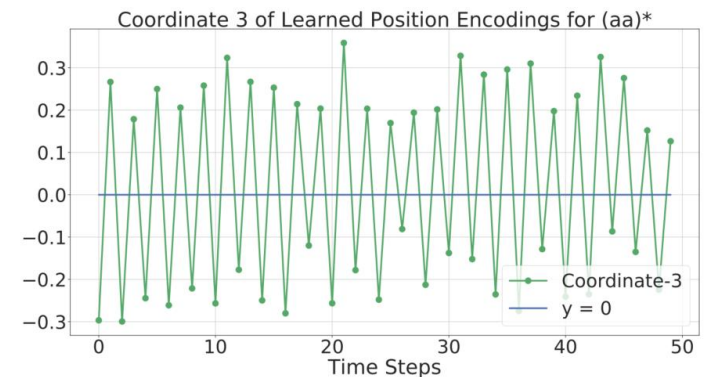


Figure 6: The values of coordiante 3 of the learned position encodings on the language $(aa)^*$. The variation in the encodings resemble a periodic behaviour similar to $\cos(n\pi)$

LSTM Networks Can Perform Dynamic Counting

Thank you!

